Start Assignment

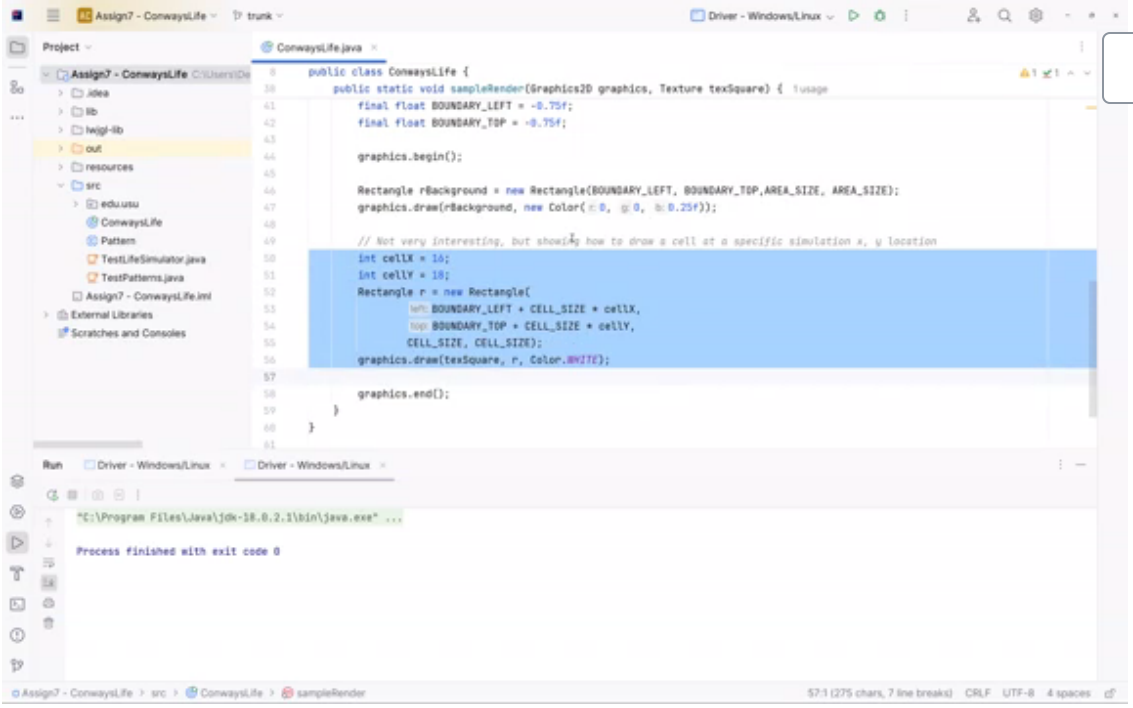
- Due Saturday by 11:59pm
- Points 40
- Submitting a file upload
- File Types zip
- Available Jan 5 at 12am - Apr 6 at 11:59pm

7 Assignment 7 - Conway's Life

Computer SCIENCE

IMPORTANT: Please review the statement on Academic Integrity. Submitting code, without permission, that is not your own may result in an automatic F for the class: Statement on Academic Integrity (https://usu.instructure.com/courses/797539/pages/statement-on-academic-integrity)

The purpose of this assignment is to give you more experience in developing Java classes, taking advantage of inheritance and polymorphism through inheritance. The subject you'll be using for this assignment is Conway's Game of Life (**wiki (https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life)**). You'll write a simulator that updates the state of a word according to a set of rules and then render that state to the console, animating it over time. I think you'll enjoy this, it is fun to play around with once it is up and going.
Please watch the following video that provides an overview of the assignment and key details for how to write the code.



19:55/23:26

Assignment

Write a program that provides an interesting animation using Conway's Game of Life rules and using at least the required patterns indicated below.

Required Patterns

- Acorn
- Block
- Blinker
- Glider

All of these patterns are shown in the wiki: **link (https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life)**

All patterns must be derived from the following class.

```
public abstract class Pattern {
    public abstract int getSizeX();
    public abstract int getSizeY();
    public abstract boolean getCell(int x, int y);
}
```

- `getSizeX` - Returns the horizontal width (in cells) of the pattern.
- `getSizeY` - Returns the vertical height (in cells) of the pattern.
- `getCell` - Returns `true` if the cell in the pattern is filled, `false` otherwise.

The derived classes may add additional private fields and methods, but no changes to the public interface allowed. You'll name these classes:

- `PatternAcorn`
- `PatternBlock`
- `PatternBlinker`
- `PatternGlider`

Simulator

Create a simulator class, named `LifeSimulator`, that can be initialized with some state (of patterns) and then update the game of life simulation. Use the following as the (required) public interface for the class.

```
public class LifeSimulator {
    public LifeSimulator(int sizeX, int sizeY) { ... }

    public int getSizeX() { ... }
    public int getSizeY() { ... }
    public boolean getCell(int x, int y) { ... }

    public void insertPattern(Pattern pattern, int startX, int startY) { ... }
    public void update() { ... }
};
```

- The constructor accepts a `sizeX` and `sizeY` indicating the size of the world. The code I have provided passes these values into the `LifeSimulator` constructor. In the constructor, an internal 2d array (the grid) is created of that size, this is the grid used to maintain the state of the simulation.
- `insertPattern` - Adds the pattern to the world, with the upper left corner beginning at `startX` and `startY`. Adding a pattern means copying the cells from the pattern into the internal grid representation of the simulator. You don't keep references to the original patterns, you copy the cell values into the simulator. Multiple patterns can be added by repeatedly calling this method.
- `update` - Performs a single step update of the world, following the four rules specified in the wiki article. The following steps provide an overview for how to do an update:
    1. Make a new grid (array) that is the same size as the current grid
        - You'll have a reference called something like 'original' and a reference called 'updated'. Where 'original' refers to the current grid state and 'updated' refers to the new grid you just created.
    2. For each cell in the original grid, use the 4 rules to decide the state of that cell in the updated grid.
    3. Put the value of the updated cell state in the updated grid. You never change any of the cell values in the original grid, you only read from it, always writing the updated values in the updated grid.
    4. When finished going through all the cells and setting their values in the updated grid, set the reference of 'original' to refer to the 'updated' grid.

- `getSizeX` & `getSizeY` - Return the size of the world.
- `getCell` - Returns `true` if the world cell is alive, `false` otherwise.

You may add any private fields and methods you like, but can not modify the public interface in any way.

Other notes

- Cells on the edge of the field should count cells past the edge (those that don't exist) as dead cells; in other words, they don't count.
  - If you want to be cool, feel free to "wrap" and count the cells from the other edge. This is definitely not required.

Renderer

Write a `render` method in the `ConwaysLife` class that takes a `LifeSimulator` and renders it (described below).

```
public static void render(Graphics2D graphics, Texture texSquare, LifeSimulator simulation) { ...
}
```

Rendering

The provided code framework utilizes a third-party framework called the **Lightweight Java Game Library (https://www.lwjgl.org/)** (LWJGL), along with an interface I have written to make using it much easier.

The `ConwaysLife.java` file has a basic demonstration for how to draw cells to the window. For each cell you want to draw, create a rectangle and then call `graphics.draw(...)` to render. You'll need to loop through all the cells in the simulator and draw each cell, each time the `render` method is called.

Animation

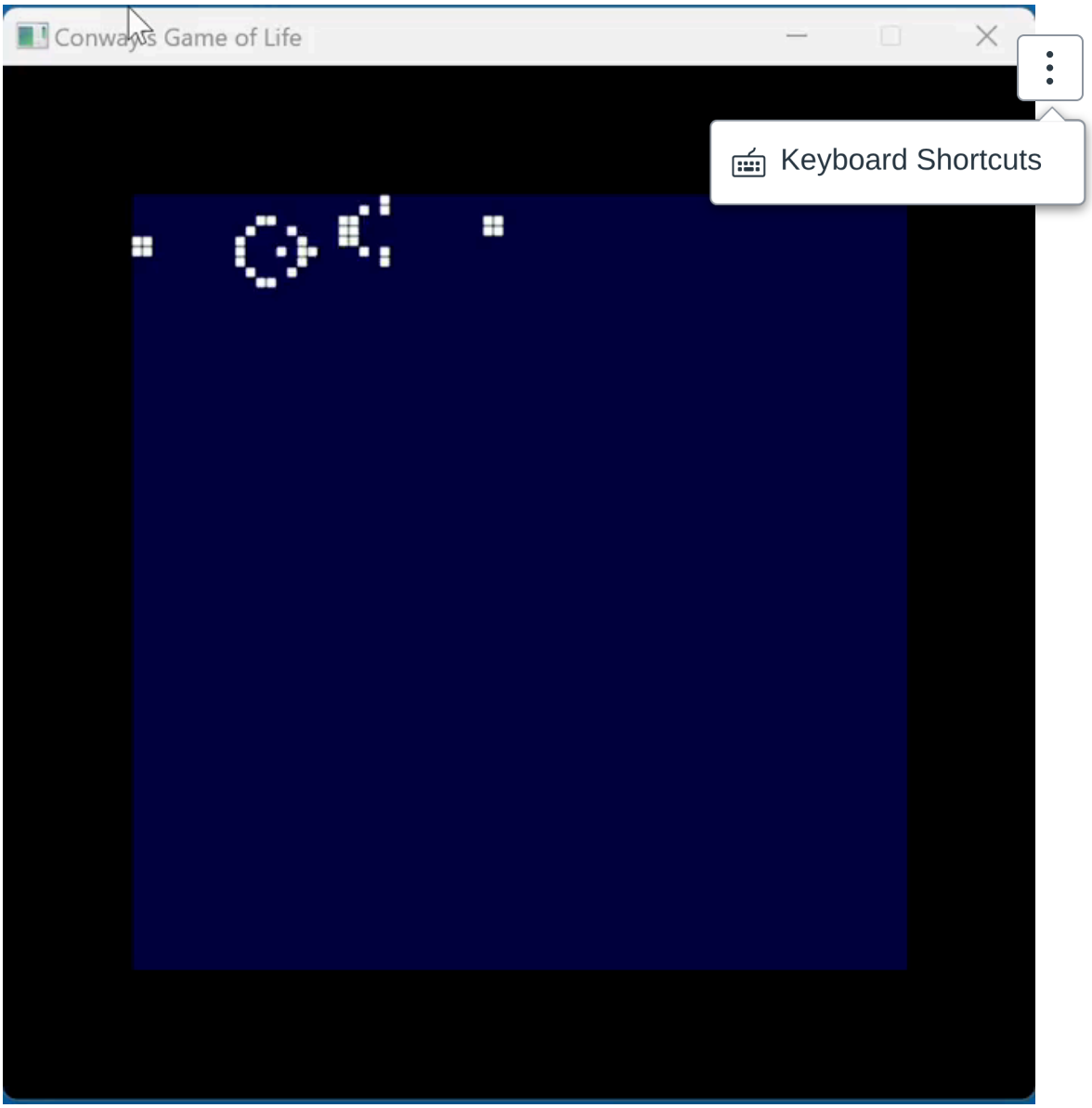
Your program will run until the ESC (escape) key is pressed. Once the programs runs, the simulation and rendering will continue until the user presses the ESC key; that is what we want.

To add a pattern to your simulator, for your animation demonstration, you'll have code something along these lines...

```
LifeSimulator mySim = new LifeSimulator(...);
mySim.insertPattern(new PatternBlock(), 0, 0);
mySim.insertPattern(new PatternBlinker(), 0, 10);
mySim.insertPattern(new PatternGlider(), 15, 15);
```

Example Animation

The following shows an example of my program running with a Gosper Glider Gun



0:00/0:20

Unit Tests

There are unit tests associated with this assignment. The provided project includes them, but they are commented out so that the sample code will compile and run. As you work on your code, you'll want to uncomment the unit tests to ensure you are passing them and implementing the logic correctly.

Notes & Submission

Use the following filenames for your code. If you have additional code files due to other patterns, that is fine, but please following the same naming convention for the files and class names (consistency is king!).

- Start with the following code framework for your project: **link (https://usu.instructure.com/courses/797539/files/98019885?verifier=pcomLIBlR6ZHvKgRgL9xXnF25du46KI2QXWtxFuL&wrap=1)** **(https://usu.instructure.com/courses/797539/files/98019885/download?verifier=pcomLIBlR6ZHvKgRgL9xXnF25du46KI2QXWtxFuL&download_frd=1)**
- Submit all of these files (and any others you create) in a single zip file. Go inside the /src folder and make a .zip file of everything in there.
  - **ConwaysLife.java**
  - **LifeSimulator.java**
  - Pattern Files
    - **Pattern.java**
    - **PatternAcorn.java**
    - **PatternBlinker.java**
    - **PatternBlock.java**
    - **PatternGlider.java**
- Your code should pass all provided unit tests (see grading rubric).
- Your code must compile without any warnings or compiler errors. See syllabus regarding code that has compiler errors.
- Your code must adhere to the CS 1410 coding standard: **link (https://usu.instructure.com/courses/797539/pages/cs-1410-coding-standard)**
- Java SDK Docs: **link (https://docs.oracle.com/en/java/javase/25/index.html)**

Assignment 7 - Rubric

Criteria	Ratings		Pts
Compiles without any warnings	**2 pts** **Full Marks**	**0 pts** **No Marks**	2 pts
Follows required course code style	**2 pts** **Full Marks**	**0 pts** **No Marks**	2 pts
Passes all unit tests	**2 pts** **Full Marks**	**0 pts** **No Marks**	2 pts
Correctly implemented render method	**5 pts** **Full Marks**	**0 pts** **No Marks**	5 pts
Correctly implemented LifeSimulator class.	**10 pts** **Full Marks**	**0 pts** **No Marks**	10 pts
PatternBlock	**2 pts** **Full Marks**	**0 pts** **No Marks**	2 pts
PatternBlinker	**2 pts** **Full Marks**	**0 pts** **No Marks**	2 pts
PatternGlider	**2 pts** **Full Marks**	**0 pts** **No Marks**	2 pts
PatternAcorn	**3 pts** **Full Marks**	**0 pts** **No Marks**	3 pts
Animation demonstration	**10 pts** **Full Marks**	**0 pts** **No Marks**	10 pts
			Total Points: 40